Lab Final

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	40
Understanding/Analysis	10					
Implementation	15					
Accuracy	10					
Task Efficiency	5					
					Total obtained mark	
Comments						

Semester: Summer 2026

Student Name: Abid Hasan Tanvir

Student ID: 253-35-326

Batch: 46

Course Code: SE132 **Course Name: Data Structure Lab**

Course Teacher Name: Tahmina Meem

Designation: Lecturer

Submission Date: 24-08-2026

Multi-Stop Bus Reservation System

GitHub Link: https://github.com/tanvirIzBack/Multi-Stop-Bus-Reservation-System.git

Novelty

Most bus ticket reservation projects treat a seat as occupied for the entire trip once booked. This project introduces **segment-aware seat sharing**: each seat can be booked by multiple passengers travelling different, non-overlapping stretches of the same route (for example, one passenger from Dhaka to Comilla, and another from Comilla to Chittagong on the same physical seat). Availability is decided using an interval-overlap check rather than a simple booked/not-booked flag, which increases how many passengers a single bus can serve without adding a single extra seat.

Which Data Structure Used, Why?

Array (seats[]): The number of seats on a bus is fixed and known in advance, so seats are stored in a fixed-size array. This gives O(1) direct access to any seat by its seat number, which is used constantly across booking, cancelling, and searching.

Linked List (per-seat bookings): Each seat can be booked by a variable number of passengers depending on how many non-overlapping segments exist, so a linked list is used instead of a fixed-size array per seat. New bookings are inserted at the head in O(1) time, which is appropriate since the list is unordered and must be fully scanned on every read regardless of insertion position.

Queue (waiting list): Passengers who cannot be seated must be served strictly in the order they arrived, which is exactly what a queue guarantees. It is implemented as a linked list with front and rear pointers, giving O(1) enqueue and O(1) dequeue.

Interval Overlap Check: Seat availability for a requested segment is decided using the standard interval-overlap rule: two segments [s1,e1) and [s2,e2) conflict only if $s1 < e2$ and $s2 < e1$. This check replaces what would otherwise require a much larger seat-by-stop occupancy table.

Code

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

#define TOTAL_SEATS 3
#define NUM_STOPS 4
const char *stopNames[NUM_STOPS] = {"Cox's Bazar", "Chittagong",
"Comilla", "Dhaka"};

// Each seat holds a LINKED LIST of segment bookings
// A seat can carry multiple passengers, as long as their segments
don't overlap.
struct Booking {
    int start;
    int end;
    char name[50];
    struct Booking *next;
};

struct Seat {
    int seatNo;
    struct Booking *bookings;  // Each seat contains a booking list
(Linked List)
}seats[TOTAL_SEATS];

struct QNode {
    char name[50];
    int start;
    int end;
    struct QNode *next;
}*front = NULL, *rear = NULL;

void initializeSeats() {
    for (int i = 0; i < TOTAL_SEATS; i++) {
        seats[i].seatNo = i + 1;
        seats[i].bookings = NULL;
    }
}

void enqueueWait(char *name, int start, int end) {
    struct QNode *node = (struct QNode*) malloc(sizeof(struct
QNode));
    strcpy(node->name, name);
    node->start = start;
    node->end = end;
    node->next = NULL;
    if(rear == NULL){
        front = rear = node;
```

```
    }else {
        rear->next = node;
        rear = node;
    }
}

int dequeueWait(char *name, int *start, int *end) {
    if (front == NULL){
         return 0;
    }
    struct QNode *node = front;
    strcpy(name, node->name);
    *start = node->start;
    *end = node->end;
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(node);
    return 1;
}

// Two segments [s1,e1) and [s2,e2) overlap if one starts before
the other ends
int segmentsOverlap(int s1, int e1, int s2, int e2) {
    return (s1 < e2 && s2 < e1);
}

int isSeatFreeForSegment(int seatIndex, int start, int end) {
    struct Booking *b = seats[seatIndex].bookings;
    while (b != NULL) {
        if (segmentsOverlap(b->start, b->end, start, end)){
             return 0;
        }
        b = b->next;
    }
    return 1;   // no existing booking conflicts with this segment
}

void addBooking(int seatIndex, int start, int end, char *name) {
    struct Booking *node = (struct Booking*) malloc(sizeof(struct
Booking));
    node->start = start;
    node->end = end;
    strcpy(node->name, name);
    node->next = seats[seatIndex].bookings;   // insert at head
    seats[seatIndex].bookings = node;
}

void removeBookingByName(int seatIndex, char *name, int
*foundStart, int *foundEnd) {
```

```
    struct Booking *cur = seats[seatIndex].bookings;
    struct Booking *prev = NULL;
    *foundStart = -1;
    *foundEnd = -1;
    while (cur != NULL) {
        if (strcmp(cur->name, name) == 0) {
            *foundStart = cur->start;
            *foundEnd = cur->end;
            if (prev == NULL) seats[seatIndex].bookings = cur-
>next;
            else prev->next = cur->next;
            free(cur);
            return;
        }
        prev = cur;
        cur = cur->next;
    }
}

void printStops() {
    printf("Available stops:\n");
    for (int i = 0; i < NUM_STOPS; i++) {
        printf("%d. %s\n", i, stopNames[i]);
    }
}

/* ---------- Menu operations ---------- */
void bookSeat() {
    char name[50];
    int start, end;
    printf("Enter passenger name: ");
    scanf(" %[^\n]", name);
    printStops();
    printf("Boarding stop number: ");
    scanf("%d", &start);
    printf("Destination stop number: ");
    scanf("%d", &end);
    if (start < 0 || end >= NUM_STOPS || start >= end) {
        printf("Invalid stop range.\n");
        return;
    }
    for (int i = 0; i < TOTAL_SEATS; i++) {
        if (isSeatFreeForSegment(i, start, end)) {
            addBooking(i, start, end, name);
            printf("Ticket booked for %s on Seat %d (%s -> %s).\n",
name, seats[i].seatNo, stopNames[start], stopNames[end]);
            return;
        }
    }
    printf("No seat available for that segment. Adding %s to the
waiting list.\n", name);
```

```
    enqueueWait(name, start, end);
}

void cancelSeat() {
    int seatNo, seatIndex;
    char name[50];
    printf("Enter seat number to cancel from: ");
    scanf("%d", &seatNo);
    if (seatNo < 1 || seatNo > TOTAL_SEATS) {
        printf("Invalid seat number.\n");
        return;
    }
    seatIndex = seatNo - 1;
    if (seats[seatIndex].bookings == NULL) {
        printf("Seat %d has no active bookings.\n", seatNo);
        return;
    }
    printf("Current bookings on Seat %d:\n", seatNo);
    struct Booking *b = seats[seatIndex].bookings;
    while (b != NULL) {
        printf(" - %s (%s -> %s)\n", b->name, stopNames[b->start],
stopNames[b->end]);
        b = b->next;
    }
    printf("Enter passenger name to cancel: ");
    scanf(" %[^\n]", name);
    int freedStart, freedEnd;
    removeBookingByName(seatIndex, name, &freedStart, &freedEnd);
    if (freedStart == -1) {
        printf("No booking found for %s on Seat %d.\n", name,
seatNo);
        return;
    }
    printf("Booking for %s (%s -> %s) on Seat %d cancelled.\n",
name, stopNames[freedStart], stopNames[freedEnd], seatNo);
    struct QNode *ptr;
    while(ptr!=NULL){
        ptr=ptr->next;
        if(front != NULL && isSeatFreeForSegment(seatIndex, front-
>start, front->end)){
            char waitName[50];
            int ws, we;
            dequeueWait(waitName, &ws, &we);
            addBooking(seatIndex, ws, we, waitName);
            printf("Seat %d now booked for %s (%s -> %s) from
waiting list.\n", seatNo, waitName, stopNames[ws], stopNames[we]);
            ptr=front;
        }
    }
}
```

```
void checkSeat() {
    int seatNo, seatIndex;
    printf("Enter seat number to check: ");
    scanf("%d", &seatNo);
    if (seatNo < 1 || seatNo > TOTAL_SEATS) {
        printf("Invalid seat number.\n");
        return;
    }
    seatIndex = seatNo - 1;
    if (seats[seatIndex].bookings == NULL) {
        printf("Seat %d is fully free for the whole route.\n",
seatNo);
        return;
    }
    printf("Seat %d bookings:\n", seatNo);
    struct Booking *b = seats[seatIndex].bookings;
    while (b != NULL) {
        printf(" - %s: %s -> %s\n", b->name, stopNames[b->start],
stopNames[b->end]);
        b = b->next;
    }
}

void waitingList() {
    printf("--- Waiting List ---\n");
    if (front == NULL) {
        printf("(empty)\n");
        return;
    }
    struct QNode *n = front;
    int i = 1;
    while (n != NULL) {
        printf("%d. %s (%s -> %s)\n", i, n->name, stopNames[n-
>start], stopNames[n->end]);
        n = n->next;
        i++;
    }
}

void display() {
    printf("Seat summary:\n");
    for (int i = 0; i < TOTAL_SEATS; i++) {
        int cnt = 0;
        struct Booking *b = seats[i].bookings;
        while (b != NULL){
            cnt++;
            b = b->next;
        }
        printf("Seat %d: %d booking(s)\n", seats[i].seatNo, cnt);
    }
    printf("\n");
```

```
}

int main() {
    initializeSeats();
    int choice = -1;
    while (choice != 6) {
        printf("==== Multi-Stop Bus Reservation System ====\n\n");
        display();
        printf("1. Book a Ticket\n");
        printf("2. Cancel a Ticket\n");
        printf("3. Check Seat Bookings\n");
        printf("4. View Waiting List\n");
        printf("5. View Route Stops\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: bookSeat();
            break;
            case 2: cancelSeat();
            break;
            case 3: checkSeat();
            break;
            case 4: waitingList();
            break;
            case 5: printStops();
            break;
            case 6: printf("Exiting system. Thank you!\n");
            break;
            default: printf("Invalid choice. Try again.\n");
        }
        printf("\n\n");
    }
    return 0;
}
```

Program Output

Console | I/O | AI Agent

```
==== Multi-Stop Bus Reservation System ====

Seat summary:
Seat 1: 0 booking(s)
Seat 2: 0 booking(s)
Seat 3: 0 booking(s)

1. Book a Ticket
2. Cancel a Ticket
3. Check Seat Bookings
4. View Waiting List
5. View Route Stops
6. Exit
Enter your choice: 1
Enter passenger name: Tanvir
Available stops:
0. Cox's Bazar
1. Chittagong
2. Comilla
3. Dhaka
Boarding stop number: 0
Destination stop number: 3
Ticket booked for Tanvir on Seat 1 (Cox's Bazar -> Dhaka).


==== Multi-Stop Bus Reservation System ====

Seat summary:
Seat 1: 1 booking(s)
Seat 2: 0 booking(s)
Seat 3: 0 booking(s)

1. Book a Ticket
2. Cancel a Ticket
3. Check Seat Bookings
4. View Waiting List
5. View Route Stops
6. Exit
Enter your choice:
```